

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计

批改人：

实验	基于 MPI 的并行矩阵乘法（进阶）	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/4/7

1. 实验目的

利用 MPI 的其他通信方式实现并行矩阵乘法，并进行对比。并尝试不同的数据/任务划分方式、提升稀疏矩阵和瘦长矩阵等特殊矩阵的矩阵乘法性能的实现方法。

2. 实验过程和核心代码

2.1 MPI 集合通信实现并行矩阵乘法

首先使用 MPI 集合通信方式实现并行矩阵乘法。我将在作业 2 的点对点通信的基础上进行修改，依旧使用行划分的方式，使用 MPI_Bcast 进行 m 、 n 、 k 矩阵维度变量的广播，通过 MPI_Scatter 进行矩阵的数据分发，并通过 MPI_Gather 进行数据收集。核心代码如下：

```

int main(int argc, char** argv) {
    int rank, size;
    Matrix A, B, C;
    double start_time, end_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    srand(time(NULL) + rank);

    if (rank == 0) {
        if (argc != 4) {
            fprintf(stderr, "用法: %s <m> <n> <k>\n", argv[0]);
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
        int m = atoi(argv[1]);
        int n = atoi(argv[2]);
        int k = atoi(argv[3]);

        initMatrix(&A, m, n);
        initMatrix(&B, n, k);
        C.rows = m;
        C.cols = k;
        C.data = (double*)malloc(m * k * sizeof(double));
    }

    start_time = MPI_Wtime();

    MPI_Bcast(&A.rows, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&A.cols, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&B.cols, 1, MPI_INT, 0, MPI_COMM_WORLD);

```

```

// 为非根进程分配内存,因为没有广播B.rows,图省事, 使用A.cols也是一样的
if (rank != 0) {
    int rows_per_proc = A.rows / size;
    A.data = (double*)malloc(rows_per_proc * A.cols * sizeof(double));
    B.data = (double*)malloc(A.cols * B.cols * sizeof(double));
    C.data = (double*)malloc(rows_per_proc * B.cols * sizeof(double));
}

// 广播矩阵B,因为没有广播B.rows,所以使用A.cols也是一样的
MPI_Bcast(B.data, A.cols * B.cols, MPI_DOUBLE, 0, MPI_COMM_WORLD);

int rows_per_proc = A.rows / size;
MPI_Scatter(A.data, rows_per_proc * A.cols, MPI_DOUBLE,
           A.data, rows_per_proc * A.cols, MPI_DOUBLE,
           0, MPI_COMM_WORLD);

Matrix local_A = {rows_per_proc, A.cols, A.data};
Matrix local_C = {rows_per_proc, B.cols, C.data};
matrixMultiply(&local_A, &B, &local_C);

MPI_Gather(C.data, rows_per_proc * B.cols, MPI_DOUBLE,
           C.data, rows_per_proc * B.cols, MPI_DOUBLE,
           0, MPI_COMM_WORLD);

end_time = MPI_Wtime();

```

实验部分结果如下:

```

cake@CAK:~/并行程序设计/hw3$ mpicc -o matrix_multiply matrix_multiply.c
cake@CAK:~/并行程序设计/hw3$ mpiexec -np 8 ./matrix_multiply 1024 1024 1024
进程数: 8, 规模: 1024
矩阵乘法计算完成!
计算时间: 0.832803 秒

```

2.2 使用 mpi_type_create_struct 聚合 MPI 进程内变量后通信

在 2.1 的代码基础上进行修改,使用 `MPI_Type_create_struct` 将矩阵的维度变量(如 `m`、`n`、`k`)打包成一个自定义的 MPI 数据类型,从而可以在一次广播中将所有维度信息一并传输,而无需进行多次 `MPI_Bcast` 调用。核心代码如下:

```
// 定义传输用的结构体 (用于打包 m, n, k)
typedef struct {
    int m;
    int n;
    int k;
} MatrixDim;
```

```
// 定义 MPI 类型来广播 dims (m, n, k)
MPI_Datatype MPI_MatrixDim;
int block_lengths[3] = {1, 1, 1};
MPI_Aint displacements[3];
MPI_Aint base;
MPI_Get_address(&dims, &base);
MPI_Get_address(&dims.m, &displacements[0]);
MPI_Get_address(&dims.n, &displacements[1]);
MPI_Get_address(&dims.k, &displacements[2]);
for (int i = 0; i < 3; i++) displacements[i] -= base;
MPI_Datatype types[3] = {MPI_INT, MPI_INT, MPI_INT};
MPI_Type_create_struct(3, block_lengths, displacements, types, &MPI_MatrixDim);
MPI_Type_commit(&MPI_MatrixDim);

// 根进程处理命令行参数并初始化矩阵
if (rank == 0) {
    if (argc != 4) {
        fprintf(stderr, "用法: %s <m> <n> <k>\n", argv[0]);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    dims.m = atoi(argv[1]);
    dims.n = atoi(argv[2]);
    dims.k = atoi(argv[3]);

    initMatrix(&A, dims.m, dims.n);
    initMatrix(&B, dims.n, dims.k);
    C.rows = dims.m;
    C.cols = dims.k;
    C.data = (double*)malloc(dims.m * dims.k * sizeof(double));
}

// 广播结构体 dims (聚合 m, n, k)
MPI_Bcast(&dims, 1, MPI_MatrixDim, 0, MPI_COMM_WORLD);
```

部分运行结果如下:

```
cake@CAK:~/并行程序设计/hw3$ mpiexec -np 8 ./matrix_multiply2 1024 1024 1024
进程数: 8, 规模: 1024
矩阵乘法计算完成!
计算时间: 0.794954 秒
```


2.3 不同的数据划分方式的尝试，以及对稀疏矩阵，瘦高矩阵特殊情况的处理

之前使用的都是对某一个矩阵行划分，现在我尝试实现两个矩阵都进行块划分，即将矩阵以块为单位进行划分，A 矩阵按行分块，B 矩阵按列分块，分配给不同的线程进行运算。主要实现是通过 `MPI_Cart_create` 将进程重组为二维进程网格，并利用 `MPI_Cart_coords` 获取当前进程在网格中的位置，对矩阵进行分块后进行分配，计算后回收，我各自实现了点对点通信和集体通信的版本核心代码如下：

点对点通信：

```
int main(int argc, char* argv[]) {
    int rank, size;
    int m, n, k;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc != 6) {
        if (rank == 0) fprintf(stderr, "Usage: %s <m> <n> <k> <dims0> <dims1>\n", argv[0]);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    m = atoi(argv[1]);
    n = atoi(argv[2]);
    k = atoi(argv[3]);

    int dims[2];
    dims[0] = atoi(argv[4]);
    dims[1] = atoi(argv[5]);
    if (dims[0] * dims[1] != size) {
        if (rank == 0) printf("Error: Number of processes must be a perfect square.\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int coords[2], periods[2] = {0, 0};
    MPI_Comm grid_comm;
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid_comm);
    MPI_Cart_coords(grid_comm, rank, 2, coords);
}
```

```

int row_block = m / dims[0];
int col_block = k / dims[1];
int inner_dim = n;

Matrix A_block = {row_block, inner_dim, malloc(row_block * inner_dim * sizeof(double))};
Matrix B_block = {inner_dim, col_block, malloc(inner_dim * col_block * sizeof(double))};
Matrix C_block = {row_block, col_block, malloc(row_block * col_block * sizeof(double))};
double start = MPI_Wtime();
// 根进程生成完整矩阵并分发

if (rank == 0) {
    Matrix A = {m, n, malloc(m * n * sizeof(double))};
    Matrix B = {n, k, malloc(n * k * sizeof(double))};
    initMatrix(&A, m, n);
    initMatrix(&B, n, k);
    // 这里感觉会耗时, 不过应该可以忽略
    // printMatrix("A", &A);
    // printMatrix("", &B);

    // 分发子块

```

```

for (int proc = 0; proc < size; proc++) {
    int p_coords[2];
    MPI_Cart_coords(grid_comm, proc, 2, p_coords);
    int row_offset = p_coords[0] * row_block;
    int col_offset = p_coords[1] * col_block;

    if (proc == 0) {
        // 当前进程保留自己的子块
        for (int i = 0; i < row_block; i++)
            for (int j = 0; j < inner_dim; j++)
                A_block.data[i * inner_dim + j] = A.data[(row_offset + i) * n + j];
        for (int i = 0; i < inner_dim; i++)
            for (int j = 0; j < col_block; j++)
                B_block.data[i * col_block + j] = B.data[i * k + col_offset + j];
    } else {
        // 发送子块
        double* A_sub = malloc(row_block * inner_dim * sizeof(double));
        double* B_sub = malloc(inner_dim * col_block * sizeof(double));

        for (int i = 0; i < row_block; i++)
            for (int j = 0; j < inner_dim; j++)
                A_sub[i * inner_dim + j] = A.data[(row_offset + i) * n + j];
        for (int i = 0; i < inner_dim; i++)
            for (int j = 0; j < col_block; j++)
                B_sub[i * col_block + j] = B.data[i * k + col_offset + j];

        MPI_Send(A_sub, row_block * inner_dim, MPI_DOUBLE, proc, 0, MPI_COMM_WORLD);
        MPI_Send(B_sub, inner_dim * col_block, MPI_DOUBLE, proc, 1, MPI_COMM_WORLD);

        free(A_sub);
    }
}

```

```

    } else {
        MPI_Recv(A_block.data, row_block * inner_dim, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(B_block.data, inner_dim * col_block, MPI_DOUBLE, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    matrixMultiply(&A_block, &B_block, &C_block);

    if (rank == 0) {
        Matrix C = {m, k, malloc(m * k * sizeof(double))};
        for (int i = 0; i < row_block; i++)
            for (int j = 0; j < col_block; j++)
                C.data[i * k + j] = C_block.data[i * col_block + j];

        for (int proc = 1; proc < size; proc++) {
            int coords[2];
            MPI_Cart_coords(grid_comm, proc, 2, coords);
            int row_offset = coords[0] * row_block;
            int col_offset = coords[1] * col_block;

            double* subblock = malloc(row_block * col_block * sizeof(double));
            MPI_Recv(subblock, row_block * col_block, MPI_DOUBLE, proc, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            for (int i = 0; i < row_block; i++)
                for (int j = 0; j < col_block; j++)
                    C.data[(row_offset + i) * k + col_offset + j] = subblock[i * col_block + j];
            free(subblock);
        }
        double end = MPI_Wtime();
        printf("计算完成, 耗时 %.6f 秒\n", end - start);
        // printMatrix("结果矩阵 C", &C);
    }
}

```

集体通信实现如下：

```

int main(int argc, char* argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc != 6) {
        if (rank == 0)
            fprintf(stderr, "Usage: %s <m> <n> <k> <dims0> <dims1>\n", argv[0]);
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int m = atoi(argv[1]);
    int n = atoi(argv[2]);
    int k = atoi(argv[3]);
    int dims[2] = {atoi(argv[4]), atoi(argv[5])};

    if (dims[0] * dims[1] != size) {
        if (rank == 0)
            fprintf(stderr, "Error: dims[0]*dims[1] != number of processes\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}

```

```

if (m % dims[0] != 0 || k % dims[1] != 0) {
    if (rank == 0)
        fprintf(stderr, "Matrix size must be divisible by dims\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
}

MPI_Comm grid_comm;
int periods[2] = {0, 0};
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &grid_comm);

int coords[2];
MPI_Cart_coords(grid_comm, rank, 2, coords);

int row_block = m / dims[0];
int col_block = k / dims[1];

Matrix A_block = {row_block, n, malloc(row_block * n * sizeof(double))};
Matrix B_block = {n, col_block, malloc(n * col_block * sizeof(double))};
Matrix C_block = {row_block, col_block, malloc(row_block * col_block * sizeof(double))};

double start = MPI_Wtime();

Matrix A_full, B_full, C_full;
if (rank == 0) {
    initMatrix(&A_full, m, n);
    initMatrix(&B_full, n, k);
}

```



```

// Scatter A
int* sendcounts_A = malloc(size * sizeof(int));
int* displs_A = malloc(size * sizeof(int));
for (int proc = 0; proc < size; ++proc) {
    int p_coords[2];
    MPI_Cart_coords(grid_comm, proc, 2, p_coords);
    int index = p_coords[0] * dims[1] + p_coords[1];
    sendcounts_A[index] = row_block * n;
    displs_A[index] = p_coords[0] * row_block * n;
}

MPI_Scatterv(rank == 0 ? A_full.data : NULL, sendcounts_A, displs_A, MPI_DOUBLE,
            A_block.data, row_block * n, MPI_DOUBLE, 0, grid_comm);

// Scatter B (column blocks)
int* sendcounts_B = malloc(size * sizeof(int));
int* displs_B = malloc(size * sizeof(int));
double* B_reordered = NULL;

if (rank == 0) {
    B_reordered = malloc(size * n * col_block * sizeof(double));
    for (int proc = 0; proc < size; ++proc) {
        int p_coords[2];
        MPI_Cart_coords(grid_comm, proc, 2, p_coords);
        int col_offset = p_coords[1] * col_block;
        int index = p_coords[0] * dims[1] + p_coords[1];

        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < col_block; ++j) {
                B_reordered[index * n * col_block + i * col_block + j] =
                    B_full.data[i * k + col_offset + j];
            }
        }
        sendcounts_B[index] = n * col_block;
        displs_B[index] = index * n * col_block;
    }
}

```

```

MPI_Scatterv(B_reordered, sendcounts_B, displs_B, MPI_DOUBLE,
            B_block.data, n * col_block, MPI_DOUBLE, 0, grid_comm);

// Local computation
matrixMultiply(&A_block, &B_block, &C_block);

// Gather C
int* recvcnts_C = malloc(size * sizeof(int));
int* displs_C = malloc(size * sizeof(int));
double* C_temp = NULL;

if (rank == 0) {
    C_full.rows = m;
    C_full.cols = k;
    C_full.data = malloc(m * k * sizeof(double));
    C_temp = malloc(size * row_block * col_block * sizeof(double));
    for (int proc = 0; proc < size; ++proc) {
        int p_coors[2];
        MPI_Cart_coords(grid_comm, proc, 2, p_coors);
        int index = p_coors[0] * dims[1] + p_coors[1];
        recvcnts_C[index] = row_block * col_block;
        displs_C[index] = index * row_block * col_block;
    }
}

MPI_Gatherv(C_block.data, row_block * col_block, MPI_DOUBLE,
            C_temp, recvcnts_C, displs_C, MPI_DOUBLE,
            0, grid_comm);

```

```

if (rank == 0) {
    for (int proc = 0; proc < size; ++proc) {
        int p_coors[2];
        MPI_Cart_coords(grid_comm, proc, 2, p_coors);
        int row_offset = p_coors[0] * row_block;
        int col_offset = p_coors[1] * col_block;
        int index = p_coors[0] * dims[1] + p_coors[1];
        for (int i = 0; i < row_block; ++i) {
            for (int j = 0; j < col_block; ++j) {
                C_full.data[(row_offset + i) * k + col_offset + j] =
                    C_temp[index * row_block * col_block + i * col_block + j];
            }
        }
    }

    double end = MPI_Wtime();
    printf("计算完成, 耗时 %.6f 秒\n", end - start);
    free(C_full.data);
    free(C_temp);
    free(B_reordered);
}

```

对于稀疏矩阵的情况，采用 CSR 格式存储稀疏矩阵，按行存储稀疏矩阵的非零数据到 values 矩阵，并且额外开放两个数组存放非零数据所在列以及每一行非零数据在 values 中起始的位置索引。通过这种方式可以极大缩小矩阵乘法空间开销以及提升乘法效率，代码如下：

```
void initSparseMatrix(SparseMatrixCSR* mat, int rows, int cols) {
    mat->rows = rows;
    mat->cols = cols;
    mat->row_ptr = malloc((rows + 1) * sizeof(int));
    int max_nnz = (int)((1.0 - SPARSITY) * rows * cols);
    mat->values = malloc(max_nnz * sizeof(double));
    mat->col_idx = malloc(max_nnz * sizeof(int));

    int nnz = 0;
    mat->row_ptr[0] = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (((double)rand() / RAND_MAX) > SPARSITY) {
                if (nnz < max_nnz) {
                    mat->values[nnz] = (double)rand() / RAND_MAX;
                    mat->col_idx[nnz] = j;
                    nnz++;
                }
            }
        }
        mat->row_ptr[i + 1] = nnz;
    }
    mat->nnz = nnz;
}
```



```

void sparseCSRMultiply(SparseMatrixCSR* A, SparseMatrixCSR* B, SparseMatrixCSR* C) {
    int m = A->rows, k = B->cols;
    int* row_ptr = malloc((m + 1) * sizeof(int));
    int* temp_col = calloc(k, sizeof(int));
    double* temp_val = calloc(k, sizeof(double));
    int capacity = 1000;
    double* values = malloc(capacity * sizeof(double));
    int* col_idx = malloc(capacity * sizeof(int));
    int nnz = 0;

    row_ptr[0] = 0;
    for (int i = 0; i < m; i++) {
        memset(temp_val, 0, k * sizeof(double));
        memset(temp_col, 0, k * sizeof(int));
        for (int idx = A->row_ptr[i]; idx < A->row_ptr[i + 1]; idx++) {
            int a_col = A->col_idx[idx];
            double a_val = A->values[idx];
            for (int j = B->row_ptr[a_col]; j < B->row_ptr[a_col + 1]; j++) {
                int b_col = B->col_idx[j];
                double b_val = B->values[j];
                if (temp_col[b_col] == 0) temp_col[b_col] = 1;
                temp_val[b_col] += a_val * b_val;
            }
        }
        for (int j = 0; j < k; j++) {
            if (temp_col[j]) {
                if (nnz == capacity) {
                    capacity *= 2;
                    values = realloc(values, capacity * sizeof(double));
                    col_idx = realloc(col_idx, capacity * sizeof(int));
                }
                values[nnz] = temp_val[j];
                col_idx[nnz] = j;
                nnz++;
            }
        }
        row_ptr[i + 1] = nnz;
    }
}

```



```

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc != 4) {
        if (rank == 0)
            printf("Usage: %s <m> <n> <k>\n", argv[0]);
        MPI_Finalize();
        return 1;
    }

    int m = atoi(argv[1]), n = atoi(argv[2]), k = atoi(argv[3]);

    SparseMatrixCSR A, B, local_A, local_C;

    int* sendcounts = malloc(size * sizeof(int));
    int* displs = malloc(size * sizeof(int));
    int* rowcounts = malloc(size * sizeof(int));
    int* rowdispls = malloc(size * sizeof(int));
    int* start_row_ptr_vals = malloc(size * sizeof(int));

    if (rank == 0) {
        srand(time(NULL));
        initSparseMatrix(&A, m, n);
        initSparseMatrix(&B, n, k);

        for (int i = 0; i < size; ++i) {
            int rs = (m * i) / size;
            int re = (m * (i + 1)) / size;
            rowcounts[i] = re - rs;
            rowdispls[i] = rs;
            sendcounts[i] = A.row_ptr[re] - A.row_ptr[rs];
            displs[i] = A.row_ptr[rs];
            start_row_ptr_vals[i] = A.row_ptr[rs];
        }
    }
}

```

```

int main(int argc, char** argv) {
    MPI_Bcast(&B.rows, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&B.cols, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(&B.nnz, 1, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank != 0) {
        B.row_ptr = malloc((B.rows + 1) * sizeof(int));
        B.values = malloc(B.nnz * sizeof(double));
        B.col_idx = malloc(B.nnz * sizeof(int));
    }

    MPI_Bcast(B.row_ptr, B.rows + 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(B.values, B.nnz, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Bcast(B.col_idx, B.nnz, MPI_INT, 0, MPI_COMM_WORLD);

    int local_rows;
    MPI_Scatter(rowcounts, 1, MPI_INT, &local_rows, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int local_row_ptr_offset = 0;
    if (rank == 0) {
        local_row_ptr_offset = 0;
    } else {
        MPI_Recv(&local_row_ptr_offset, 1, MPI_INT, 0, 100, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }

    if (rank == 0) {
        for (int i = 1; i < size; ++i)
            MPI_Send(&start_row_ptr_vals[i], 1, MPI_INT, i, 100, MPI_COMM_WORLD);
    }

    int* local_row_ptr = malloc((local_rows + 1) * sizeof(int));
    MPI_Scatterv(A.row_ptr + 1, rowcounts, rowdispls, MPI_INT,
                local_row_ptr + 1, local_rows, MPI_INT, 0, MPI_COMM_WORLD);
    local_row_ptr[0] = 0;
    for (int i = 0; i < local_rows; ++i)
        local_row_ptr[i + 1] -= local_row_ptr_offset;
}

```

```

int local_nnz = local_row_ptr[local_rows];

double* local_values = malloc(local_nnz * sizeof(double));
int* local_col_idx = malloc(local_nnz * sizeof(int));

MPI_Scatterv(A.values, sendcounts, displs, MPI_DOUBLE,
            local_values, local_nnz, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatterv(A.col_idx, sendcounts, displs, MPI_INT,
            local_col_idx, local_nnz, MPI_INT, 0, MPI_COMM_WORLD);

local_A.rows = local_rows;
local_A.cols = n;
local_A.nnz = local_nnz;
local_A.values = local_values;
local_A.col_idx = local_col_idx;
local_A.row_ptr = local_row_ptr;

MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

sparseCSRMultiply(&local_A, &B, &local_C);

double end = MPI_Wtime();

int* all_nnz = NULL;
int* recvd displs = NULL;
if (rank == 0) {
    all_nnz = malloc(size * sizeof(int));
}

MPI_Gather(&local_C.nnz, 1, MPI_INT, all_nnz, 1, MPI_INT, 0, MPI_COMM_WORLD);

```

```

if (rank == 0) {
    recvdispls = malloc(size * sizeof(int));
    recvdispls[0] = 0;
    for (int i = 1; i < size; ++i)
        recvdispls[i] = recvdispls[i - 1] + all_nnz[i - 1];

    int total_nnz = recvdispls[size - 1] + all_nnz[size - 1];
    double* final_values = malloc(total_nnz * sizeof(double));
    int* final_col_idx = malloc(total_nnz * sizeof(int));

    MPI_Gatherv(local_C.values, local_C.nnz, MPI_DOUBLE,
                final_values, all_nnz, recvdispls, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Gatherv(local_C.col_idx, local_C.nnz, MPI_INT,
                final_col_idx, all_nnz, recvdispls, MPI_INT, 0, MPI_COMM_WORLD);

    printf("总乘法耗时: %.6f 秒\n", end - start);

    free(final_values);
    free(final_col_idx);
    free(recvdispls);
    free(all_nnz);
} else {
    MPI_Gatherv(local_C.values, local_C.nnz, MPI_DOUBLE,
                NULL, NULL, NULL, MPI_DOUBLE, 0, MPI_COMM_WORLD);
    MPI_Gatherv(local_C.col_idx, local_C.nnz, MPI_INT,
                NULL, NULL, NULL, MPI_INT, 0, MPI_COMM_WORLD);
}

free(local_A.values); free(local_A.col_idx); free(local_A.row_ptr);
free(local_C.values); free(local_C.col_idx); free(local_C.row_ptr);
free(B.values); free(B.col_idx); free(B.row_ptr);
free(sendcounts); free(displs); free(rowcounts); free(rowdispls); free(start_row_ptr_vals);

MPI_Finalize();
return 0;
}

```

对于“瘦高型矩阵”，其行数较多，而列数少，可以使用按行分块的方式进行划分，如果第二个矩阵规模较小，这种方法可以有效提高缓存命中率，这和我们初始实现的代码一致，因此不再进行额外实现。

3. 实验结果

使用 MPI 集合通信实现并行矩阵乘法结果填表如下（单位为 s）：

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.0045	0.0321	0.3799	3.2052	107.6315
2	0.0021	0.0170	0.1964	2.1212	55.0281
4	0.0077	0.0110	0.1093	1.2861	29.9919
8	0.0101	0.0335	0.1060	0.8328	21.4545

16	0.0280	0.0496	0.0935	0.6853	14.5407
----	--------	--------	--------	--------	---------

在 2.1 代码基础上，构建自定义 MPI 数据结构进行数据传递结果如下（单位为 s）：

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.0044	0.0321	0.3715	3.3194	107.0133
2	0.0020	0.0172	0.1923	1.9922	50.7277
4	0.0010	0.0101	0.0976	1.2159	29.9348
8	0.0005	0.0051	0.0637	0.7949	20.8303
16	0.0005	0.0047	0.0509	0.5560	13.1309

由于在此次实验时我使用的是 wsl，而上次实验使用的是虚拟机，因此在 wsl 上重新运行实验 2 的代码，结果如下：

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.0058	0.0426	0.4542	4.0308	117.1388
2	0.0034	0.0236	0.2457	2.4693	60.9013
4	0.0027	0.0202	0.1399	1.5706	30.2018
8	0.0317	0.0270	0.1146	0.9099	19.5635
16	0.067	0.0554	0.1162	0.8956	15.4927

先比较 MPI 点对点通信和集体通信的效率，可以发现，无论是矩阵规模大还是规模小时，集体通信都稍优于点对点通信，在矩阵规模大，进程多时，通信开销占比很小，两者差距几乎没有。整体结果体现了集体通信相对于点对点通信在通信开销上的优势。

再比较使用自定义数据结构减少广播次数所带来的性能提升，对比表 1 和表 2 可以发现，由于我们只是通过自定义数据结构优化了 m、n、k 矩阵维度的广播，占程序通信开销其实很小，因此在矩阵规模较大时是几乎没有差距的，而矩阵规模小时，这个性能提升就比较明显（可以对比矩阵规模为 128 的一列），这符合预期。

再看表 1 内部不同进程数之间的比较，可以发现规模大时进程数的增加可以显著提升性能，而当矩阵规模小时，通信开销开始占据主要开销部分，导致进程数增加时反而性能下降，所以我们需要根据矩阵规模选择合适的进程数。

这里把集体通信版本的分块划分实现的运行结果展示出来，如下：

进程数	矩阵规模				
	128	256	512	1024	2048
1	0.0049	0.0378	0.3909	3.9497	101.216022
2	0.0028	0.0194	0.1959	2.119	37.8447
4	0.0020	0.0128	0.0943	1.089	20.7848
8	0.0023	0.0115	0.0791	0.5629	7.2044
16	0.0031	0.0139	0.0718	0.5264	6.3403

对比可以明显看到，在矩阵规模很大时，分块的方法的效率远高于按行划分，分析原因可能如下：其一是分块划分是对 AB 两个矩阵都进行划分，传输开销更小。在更小的块中进行计算，cache 命中率更高。

虽然分块矩阵在处理规模大的矩阵时性能很高，但其对进程数有要求，最起码保证进程数为合数，并且最好是矩阵的规模可以整除进程重组成二维形成的维度，因此使用分块时对进程数有一定要求。

使用 CSR 处理稀疏矩阵时，当矩阵规模为 2048 时运行结果如下：

```
● cake@CAK:~/并行程序设计/hw3$ mpiexec --oversubscribe -np 16 ./csr_csr 2048 2048 2048
总乘法耗时: 0.135625 秒
● cake@CAK:~/并行程序设计/hw3$ mpiexec --oversubscribe -np 8 ./csr_csr 2048 2048 2048
总乘法耗时: 0.189707 秒
● cake@CAK:~/并行程序设计/hw3$ mpiexec --oversubscribe -np 4 ./csr_csr 2048 2048 2048
总乘法耗时: 0.270807 秒
● cake@CAK:~/并行程序设计/hw3$ mpiexec --oversubscribe -np 2 ./csr_csr 2048 2048 2048
总乘法耗时: 0.510737 秒
● cake@CAK:~/并行程序设计/hw3$ mpiexec --oversubscribe -np 1 ./csr_csr 2048 2048 2048
总乘法耗时: 0.988629 秒
```

可以看到运行时间大幅缩减，其原因也很好理解，因为 CSR 只存储非零数据，从而在计算时避免了所有的无意义的“0 操作”。并且 CSR 的存储结构节省了大量的存储空间，在空间和时间上均有巨大提升。

4. 实验感想

通过这次实验，我利用 MPI 的不同通信方式实现大规模并行矩阵乘法，并且详细分析了他们各自的特点。尝试了两个矩阵都进行划分的块划分方式，并与之前单一进行行划分的方式进行对比。实现了 CSR 存储稀疏矩阵来进行大规模

并行乘法计算。分析了瘦高矩阵合适的划分方式，即按行进行划分。通过这次实验，我对使用 MPI 进行并行计算的理解更加深入，受益匪浅。